Programs

1. Program to implement FCFS scheduling algorithm without arrival times.

Aim: To implement FCFS scheduling algorithm without Arrival times given and finding average Waiting time and average Turn around time.

Algorithm:

1.Start.

2.Read the number of processes n.

3.Enter the Burst Time (BT) for each process.

4.Set Waiting Time (WT) of the first process to 0.

5.For each remaining process:

WT[i] = WT[i-1] + BT[i-1]

6.Calculate Turnaround Time (TAT) for each process:

TAT[i] = WT[i] + BT[i]

7.Calculate:

*Average Waiting Time (AWT)

*Average Turnaround Time (ATAT)

8.Display the Process ID, Burst Time, Waiting Time, and Turnaround Time.

9.Stop.

Problem:

Consider the following process and calculate average WT and average TAT .Assume that the processes arrived in the order as follows:

Process	-	Burst Time
P1	-	2
P2	-	3
P3	-	4

Program:

```
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], wt[n], tat[n];
    float avg_wt = 0, avg_tat = 0;
    printf("Enter Burst Time for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
   for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
    }
   printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
              i + 1, bt[i], wt[i], tat[i]);
    }

  avg_wt /= n;
    avg_tat /= n;
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

OUTPUT:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

TESTCASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\" ; if ($?) { gcc FCFS.c -o FCFS } ; if ($?) { .\FCFS }
Enter number of processes: 3
Enter Burst Time for each process:
P1: 2
P2: 3
P3: 4

Process Burst Time      Waiting Time    Turnaround Time
P1      2               0               2
P2      3               2               5
P3      4               5               9

Average Waiting Time = 2.33
Average Turnaround Time = 5.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 2:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\" ; if ($?) { gcc FCF
Enter number of processes: 4
Enter Burst Time for each process:
P1: 3
P2: 4
P3: 2
P4: 1

Process Burst Time      Waiting Time    Turnaround Time
P1      3               0               3
P2      4               3               7
P3      2               7               9
P4      1               9               10

Average Waiting Time = 4.75
Average Turnaround Time = 7.25
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\
Enter number of processes: 3
Enter Burst Time for each process:
P1: 3
P2: 5
P3: 7

Process Burst Time      Waiting Time    Turnaround Time
P1      3               0               3
P2      5               3               8
P3      7               8               15

Average Waiting Time = 3.67
Average Turnaround Time = 8.67
```

2. Program to implement FCFS scheduling algorithm with arrival times.

Aim: To implement FCFS scheduling algorithm with Arrival times given and finding average Waiting time and average Turn around time.

Algorithm:

1.Start.

2.Read the number of processes n.

3.Enter the Arrival Time (AT) and Burst Time (BT) for each process.

4.Arrange the processes in ascending order of Arrival Time.

5.Set the Completion Time (CT) of the first process:

CT = AT + BT

6.For each remaining process:

If the CPU is idle (CT < AT), set CT = AT + BT.

Otherwise, CT = CT + BT.

7.Calculate:

TAT = CT - AT

WT = TAT - BT

8.Find:

Average Waiting Time (AWT)

Average Turnaround Time (ATAT)

9.Display Process ID, AT, BT, CT, WT, and TAT.

10.Stop

Problem: Calculate the average waiting time and average turn around time for processes given below and draw a gantt chart.

Process	AT	BT
P1	0	3
P2	2	6
P3	1	5

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

```
#include <stdio.h>

int main() {
   int n, i, j;
   int at[20], bt[20], ct[20], tat[20], wt[20];
   float avgWT = 0, avgTAT = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("\nEnter Arrival Time and Burst Time:\n");
   for(i = 0; i < n; i++) {
      printf("Process P%d\n", i + 1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
   }

   for(i = 0; i < n - 1; i++) {
      for(j = i + 1; j < n; j++) {
         if(at[i] > at[j]) {
            int temp;
            temp = at[i];
            at[i] = at[j];
            at[j] = temp;

            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
         }
      }
   }

   ct[0] = at[0] + bt[0];


 for(i = 1; i < n; i++) {
      if(ct[i - 1] < at[i])
```

```
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i - 1] + bt[i];
    }


    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        avgWT += wt[i];
        avgTAT += tat[i];
    }

    avgWT /= n;
    avgTAT /= n;
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avgWT);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT);

    return 0;
}
```

OUTPUT:

TEST CASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3

Enter Arrival Time and Burst Time:
Process P1
Arrival Time: 0
Burst Time: 3
Process P2
Arrival Time: 2
Burst Time: 6
Process P3
Arrival Time: 1
Burst Time: 5

P       AT      BT      CT      TAT     WT
P1      0       3       3       3       0
P2      1       5       8       7       2
P3      2       6       14      12      6

Average Waiting Time = 2.67
Average Turnaround Time = 7.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 2:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3

Enter Arrival Time and Burst Time:
Process P1
Arrival Time: 3
Burst Time: 6
Process P2
Arrival Time: 2
Burst Time: 3
Process P3
Arrival Time: 1
Burst Time: 8

P       AT      BT      CT      TAT     WT
P1      1       8       9       8       0
P2      2       3       12      10      7
P3      3       6       18      15      9

Average Waiting Time = 5.33
Average Turnaround Time = 11.00
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3

Enter Arrival Time and Burst Time:
Process P1
Arrival Time: 0
Burst Time: 4
Process P2
Arrival Time: 1
Burst Time: 3
Process P3
Arrival Time: 2
Burst Time: 6

P       AT      BT      CT      TAT     WT
P1      0       4       4       4       0
P2      1       3       7       6       3
P3      2       6       13      11      5

Average Waiting Time = 2.67
Average Turnaround Time = 7.00
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

3. Program to implement SJP scheduling algorithm in preemptive mode(SRTF).

Aim: : To implement SJF scheduling algorithm in preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Set the current time t = 0.

5 At each unit of time:

- Find all processes that have arrived ($AT \leq t$) and are not completed.
- Select the process with the smallest remaining burst time.
- If a new process arrives with a shorter remaining time, preempt the currently running process and execute the new one.

6. Reduce the selected process's remaining burst time by 1.
7. If the remaining burst time becomes 0:

Mark the process as completed.

Calculate:

- Completion Time (CT) = Current Time + 1
- Turnaround Time (TAT) = CT − AT
- Waiting Time (WT) = TAT − BT

Repeat Steps 5–7 until all processes are completed.

Calculate:

- Average Waiting Time = ΣWT / n
- Average Turnaround Time = ΣTAT / n

Stop.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Problem: Calculate the average waiting time and average turn around time for processes given below and draw a gantt chart.

Process	AT	BT
P1	2	3
P2	1	2
P3	2	3

Program:

```
#include <stdio.h>

    int main() {
        int n, i;
        int at[20], bt[20], rt[20];
        int ct[20], tat[20], wt[20];

        int time = 0;
        int completed = 0;
        int shortest;
        int min;

        float avgWT = 0, avgTAT = 0;

        printf("Enter number of processes: ");
        scanf("%d", &n);

        // Input Arrival Time and Burst Time
        for(i = 0; i < n; i++) {
            printf("Enter Arrival Time and Burst Time of P%d: ", i + 1);
            scanf("%d %d", &at[i], &bt[i]);

            // Remaining time initially equals Burst Time
            rt[i] = bt[i];

            ct[i] = 0;
            tat[i] = 0;
            wt[i] = 0;
        }

        // SJF Preemptive / SRTF
        while(completed < n) {

            shortest = -1;
            min = INT_MAX;

            // Find process with shortest remaining time
            for(i = 0; i < n; i++) {
                if(at[i] <= time && rt[i] > 0 && rt[i] < min) {
                    min = rt[i];
                    shortest = i;
                }
            }
```

```
        if(shortest == -1) { time++;
                continue;
            }
            rt[shortest]--;
            time++;
            if(rt[shortest] == 0) {
                completed++;
                ct[shortest] = time;
                tat[shortest] = ct[shortest] - at[shortest];
                wt[shortest] = tat[shortest] - bt[shortest];
                avgWT += wt[shortest];
                avgTAT += tat[shortest];
            }
        }
    printf("\n----------------------------------------------------\n");
        printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
        printf("----------------------------------------------------\n");

        for(i = 0; i < n; i++) {
            printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
        }
        printf("----------------------------------------------------\n");
        printf("Average Waiting Time = %.2f\n", avgWT / n);
        printf("Average Turnaround Time = %.2f\n", avgTAT / n);
        return 0;
}
```

OUTPUT:

TEST CASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 2
3
Enter Arrival Time and Burst Time of P2: 1
2
Enter Arrival Time and Burst Time of P3: 2
3

-----------------------------------------------------
Process AT      BT      CT      TAT     WT
-----------------------------------------------------
P1      2       3       6       4       1
P2      1       2       3       2       0
P3      2       3       9       7       4
-----------------------------------------------------
Average Waiting Time = 1.67
Average Turnaround Time = 4.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 2:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\" ;
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 1
3
Enter Arrival Time and Burst Time of P2: 2
5
Enter Arrival Time and Burst Time of P3: 4
7

-----------------------------------------------------
Process AT      BT      CT      TAT     WT
-----------------------------------------------------
P1      1       3       4       3       0
P2      2       5       9       7       2
P3      4       7       16      12      5
-----------------------------------------------------
Average Waiting Time = 2.33
Average Turnaround Time = 7.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 2
4
Enter Arrival Time and Burst Time of P2: 5
6
Enter Arrival Time and Burst Time of P3: 1
8

-----------------------------------------------------
Process AT      BT      CT      TAT     WT
-----------------------------------------------------
P1      2       4       6       4       0
P2      5       6       12      7       1
P3      1       8       19      18      10
-----------------------------------------------------
Average Waiting Time = 3.67
Average Turnaround Time = 9.67
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

4. Program to implement SJP scheduling algorithm in non-preemptive mode.

Aim: To implement SJP scheduling algorithm in non-preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Set the current time t = 0 and completed processes count = 0.
5. While all processes are not completed:
   - Find all processes that have arrived ($AT \leq t$) and are not completed.
   - Select the process with the smallest Burst Time (BT).
   - If no process has arrived, increment the current time by 1.

     Otherwise:
     - Execute the selected process until it finishes (without interruption).
     - Update:
       - Completion Time (CT) = t + BT
       - Turnaround Time (TAT) = CT − AT
       - Waiting Time (WT) = TAT − BT
     - Mark the process as completed.
     - Update the current time to the completion time.
6. Repeat Step 5 until all processes are completed.
7. Calculate:
   - Average Waiting Time = ΣWT / n
   - Average Turnaround Time = ΣTAT / n
9. Display Process ID, AT, BT, CT, TAT, and WT.
10. Stop.

Problem:

Calculate Average waiting time , Average turn around time using SRTF for the following processes given below.

Process	AT	BT
P1	2	3
P2	4	5
P3	2	3

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

```
#include <stdio.h>
int main() {
        int n, i, time = 0, completed = 0;
        int at[20], bt [20], ct[20], tat[20], wt[20];
        int visited[20] = {0};
       printf("Enter number of processes: ");
        scanf("%d", &n);

        for(i = 0; i < n; i++) {
           printf("Enter Arrival Time of P%d: ", i + 1);
           scanf("%d", &at[i]);

           printf("Enter Burst Time of P%d: ", i + 1);
           scanf("%d", &bt[i]);
        }
     while(completed < n) {
           int idx = -1;
           int minBT = 9999;

           for(i = 0; i < n; i++) {
              if(at[i] <= time && visited[i] == 0 && bt[i] < minBT) {
                 minBT = bt[i];
                 idx = i;
              }
           }
```

```
        if(idx == -1) {
            time++;


    } else {
            ct[idx] = time + bt[idx];
            time = ct[idx];
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            visited[idx] = 1;
            completed++;
        }
    }

    float avgWT = 0, avgTAT = 0;

    printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
              i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);
        avgWT += wt[i];
        avgTAT += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", avgWT / n);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

    return 0;}
```

OUTPUT:

TEST CASE 1:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\SJF_NONPREEMPTIVE.exe"
Enter number of processes (max 20): 3
Enter Arrival Time of P1: 2
Enter Burst Time of P1: 3
Enter Arrival Time of P2: 4
Enter Burst Time of P2: 5
Enter Arrival Time of P3: 2
Enter Burst Time of P3: 3

Process AT      BT      CT      WT      TAT
P1      2       3       5       0       3
P2      4       5       13      4       9
P3      2       3       8       3       6

Average Waiting Time = 2.33
Average Turnaround Time = 6.00

Process returned 0 (0x0)   execution time : 36.339 s
Press any key to continue.
```

TESTCASE 2:

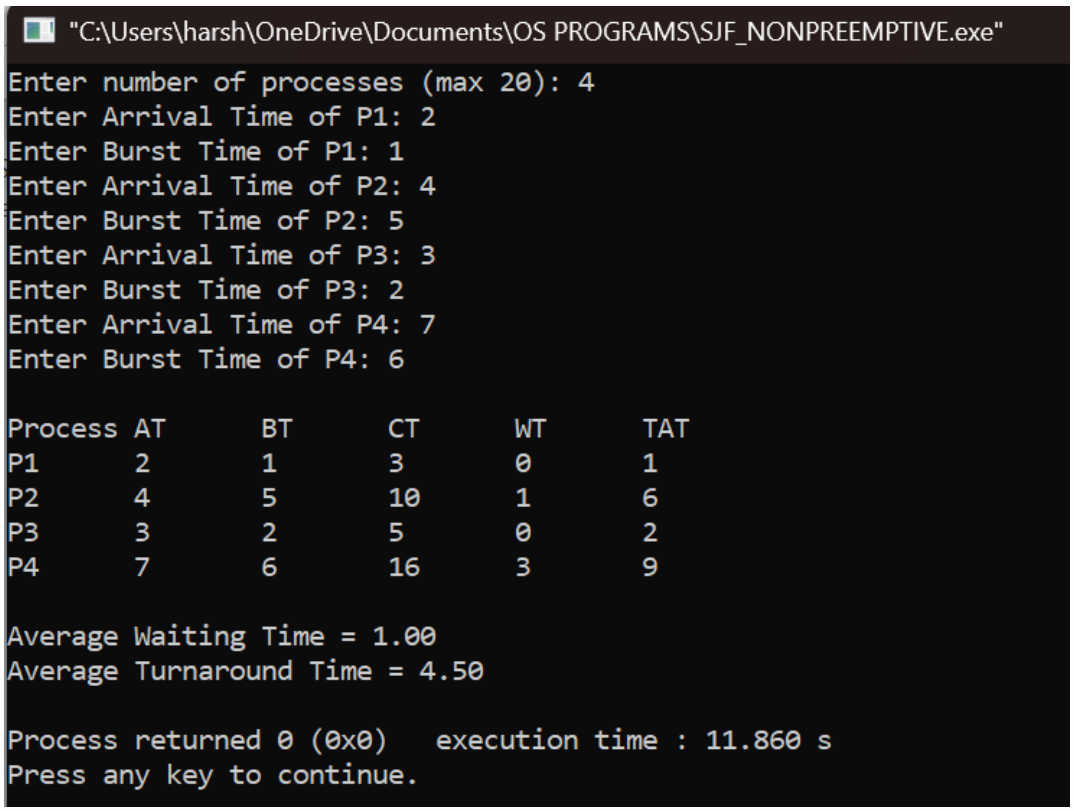
```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\SJF_NONPREEMPTIVE.exe"
Enter number of processes (max 20): 3
Enter Arrival Time of P1: 1
Enter Burst Time of P1: 2
Enter Arrival Time of P2: 2
Enter Burst Time of P2: 3
Enter Arrival Time of P3: 3
Enter Burst Time of P3: 4

Process AT      BT      CT      WT      TAT
P1      1       2       3       0       2
P2      2       3       6       1       4
P3      3       4       10      3       7

Average Waiting Time = 1.33
Average Turnaround Time = 4.33

Process returned 0 (0x0)   execution time : 9.041 s
Press any key to continue.
```

TESTCASE 3:


"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\SJF_NONPREEMPTIVE.exe"
Enter number of processes (max 20): 4
Enter Arrival Time of P1: 2
Enter Burst Time of P1: 1
Enter Arrival Time of P2: 4
Enter Burst Time of P2: 5
Enter Arrival Time of P3: 3
Enter Burst Time of P3: 2
Enter Arrival Time of P4: 7
Enter Burst Time of P4: 6
Process AT BT CT WT TAT
P1 2 1 3 0 1
P2 4 5 10 1 6
P3 3 2 5 0 2
P4 7 6 16 3 9
Average Waiting Time = 1.00
Average Turnaround Time = 4.50
Process returned 0 (0x0) execution time : 11.860 s
Press any key to continue.